

CSA0603-DESIGN AND ANALYSIS OF ALGORITHM

CO 2- ASSESSMENT 3 – DEBUGGING AND OPTIMIZATION TASK

NAME:S.SRI MATHI

REG NO:192424095

QUESTION

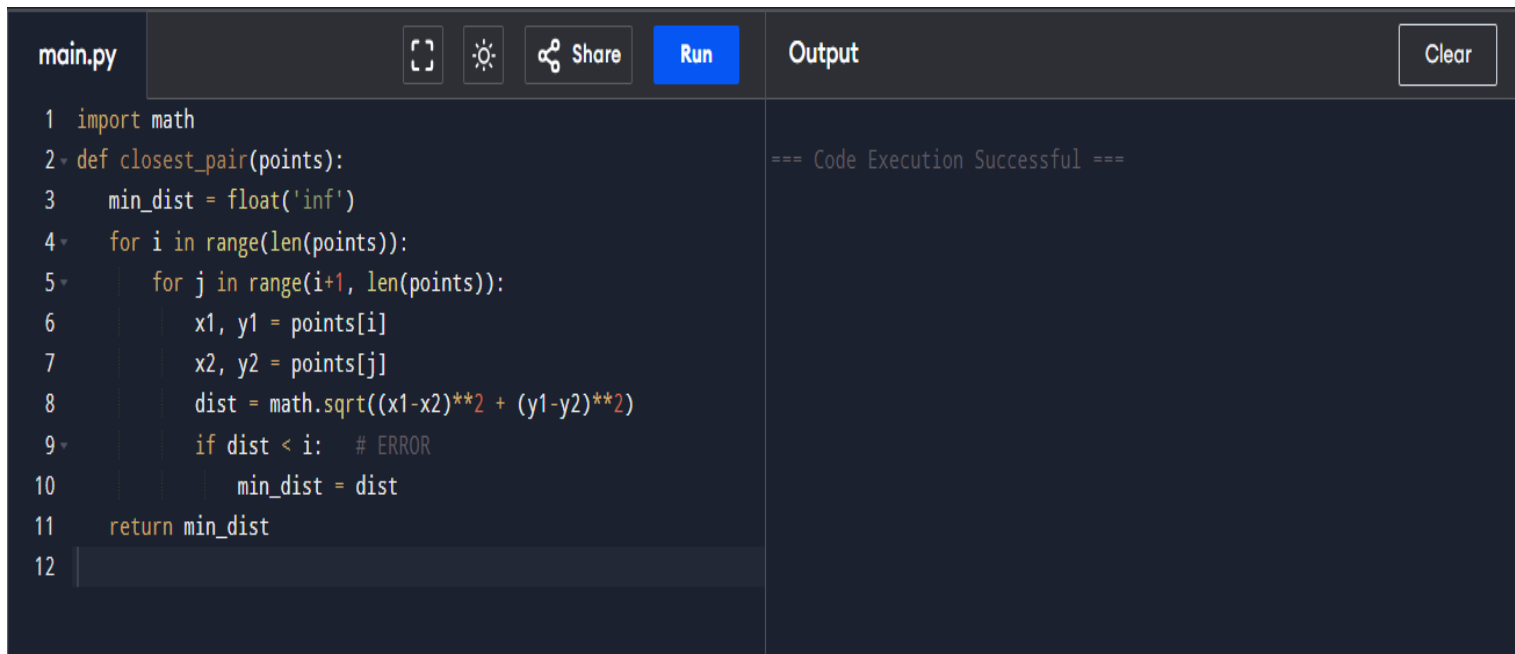
Q1. Debugging Closest Pair Code (Comparison with Wrong Variable)

The following brute force Closest Pair implementation calculates distances correctly but compares the newly computed distance against the wrong reference value. Carefully analyze the update logic and identify why the minimum pair is not always preserved. Apply debugging to ensure that the smallest distance found so far is maintained correctly throughout the nested loops. Optimize the implementation by avoiding repeated tuple indexing where possible. Analyze the time complexity of the corrected algorithm and rewrite the final version with proper comments.

```
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            x1, y1 = points[i]
            x2, y2 = points[j]
            dist = math.sqrt((x1-x2)**2 + (y1-y2)**2)
            if dist < i:
                min_dist = dist
    return min_dist
```

DEBUGGING

Step 1: Run the given code



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'closest_pair(points)' that calculates the minimum distance between any two points in a list. The function uses nested loops to compare every pair of points and calculates the distance using the Euclidean formula. The output pane shows '=== Code Execution Successful ===', indicating that the code ran without errors, but no output was produced.

```
1 import math
2 def closest_pair(points):
3     min_dist = float('inf')
4     for i in range(len(points)):
5         for j in range(i+1, len(points)):
6             x1, y1 = points[i]
7             x2, y2 = points[j]
8             dist = math.sqrt((x1-x2)**2 + (y1-y2)**2)
9             if dist < i: # ERROR
10                 min_dist = dist
11 return min_dist
12
```

Output: === Code Execution Successful ===

Observation:

The program runs without displaying an error, but **no output is produced**.

Reason identified:

The code only defines the function `def closest_pair(points):`

The function is not called, and there is no `print()` statement.

Step 2 : Add input

Adding these lines below the function

```
points = [(1, 2), (2, 3), (10, 12)]
```

```
main.py  [Full Screen] [Theme] [Share] [Run] [Output] [Clear]

1 import math
2 def closest_pair(points):
3     min_dist = float('inf')
4     for i in range(len(points)):
5         for j in range(i + 1, len(points)):
6             x1, y1 = points[i]
7             x2, y2 = points[j]
8             dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
9             if dist < i:
10                 min_dist = dist
11     return min_dist
12 points = [(1, 2), (2, 3), (10, 12)]

=== Code Execution Successful ===
```

Observation:

The answer should not be infinity because distances exist between the given points.

The distance between (1, 2) and (2, 3) is approximately 1.414

Therefore, getting inf shows that min_dist was never updated.

STEP 3: Add function call

Add this line

```
result = closest_pair(points)
```

```
print("Minimum distance:", result)
```

```
main.py  [Full Screen] [Theme] [Share] [Run] [Output] [Clear]

1 import math
2 def closest_pair(points):
3     min_dist = float('inf')
4     for i in range(len(points)):
5         for j in range(i + 1, len(points)):
6             x1, y1 = points[i]
7             x2, y2 = points[j]
8             dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
9             if dist < i:
10                 min_dist = dist
11     return min_dist
12 points = [(1, 2), (2, 3), (10, 12)]

=== Code Execution Successful ===
```

Step 4: Locate the logical error

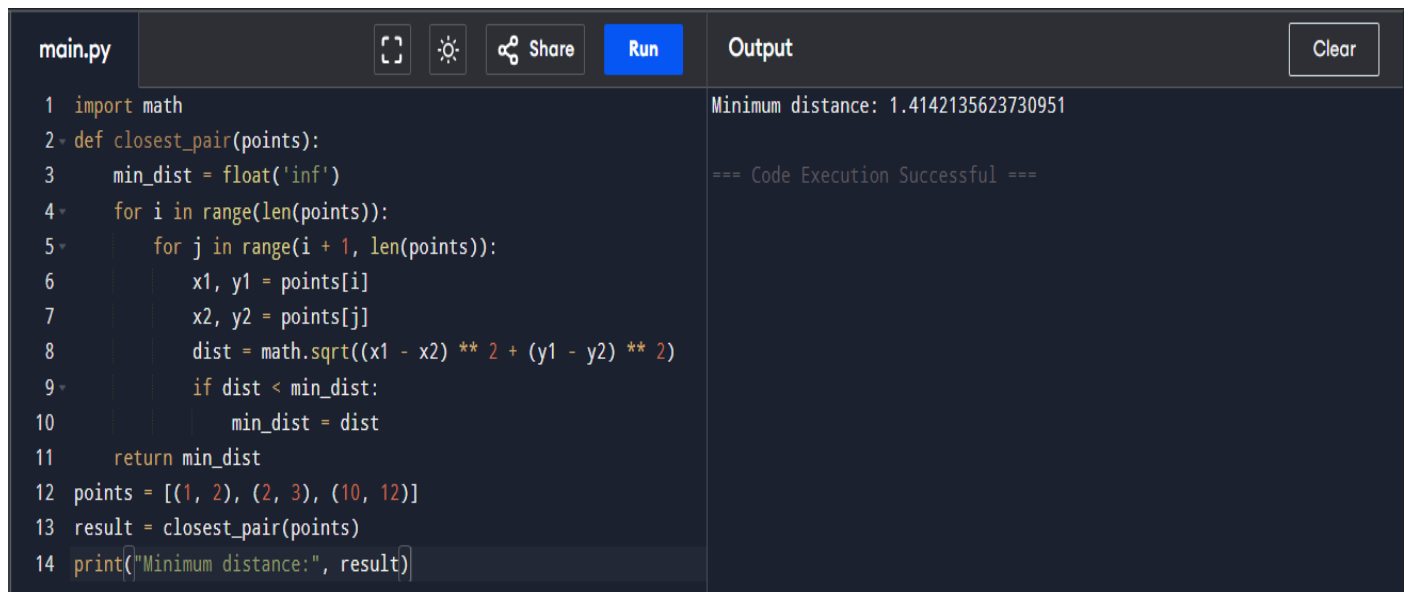
The incorrect line is: `if dist < i`

Comparing a distance with a point index is logically wrong.

The new distance must be compared with the minimum distance found so far.

Step 5: Correct the comparison line

Replace: `if dist < i` With `if dist < min_dist:`



```
main.py  [Icons] [Run] [Clear]
1 import math
2 def closest_pair(points):
3     min_dist = float('inf')
4     for i in range(len(points)):
5         for j in range(i + 1, len(points)):
6             x1, y1 = points[i]
7             x2, y2 = points[j]
8             dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
9             if dist < min_dist:
10                 min_dist = dist
11     return min_dist
12 points = [(1, 2), (2, 3), (10, 12)]
13 result = closest_pair(points)
14 print("Minimum distance:", result)
```

Output

Minimum distance: 1.4142135623730951

=== Code Execution Successful ===

Observation

The program now produces the correct minimum distance.

Step 6: Optimize repeated tuple indexing

In the current code `x1, y1 = points[i]` is placed inside the inner loop.

Move outside the inner loop

```
for i in range(len(points)):
    x1, y1 = points[i]
    for j in range(i + 1, len(points)):
        x2, y2 = points[j]
```

```
main.py  [Icons] [Share] [Run] [Clear]

3 def closest_pair(points):
4     min_dist = float('inf')
5     for i in range(len(points) - 1):
6         x1, y1 = points[i]
7         for j in range(i + 1, len(points)):
8             x2, y2 = points[j]
9             dist = math.sqrt(
10                 (x1 - x2) ** 2 + (y1 - y2) ** 2
11             )
12             if dist < min_dist:
13                 min_dist = dist
14     return min_dist
15 points = [(1, 2), (2, 3), (10, 12)]
16 result = closest_pair(points)
17 print("Minimum distance:", result)
```

Output

Minimum distance: 1.4142135623730951

=== Code Execution Successful ===

Algorithm: Brute-Force Closest Pair

Input: A list of points (x, y)

Output: The minimum distance between any two points

- Start.
- Read the list of points.
- Set min_dist to infinity.
- Select each point one by one using index i.
- Store the coordinates of the first point as x1 and y1.
- Compare it with every point after it using index j.
- Store the second point coordinates as x2 and y2.
- Calculate the Euclidean distance:
- $\text{distance} = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$
- Compare the calculated distance with min_dist.
- If the new distance is smaller, update min_dist.
- Repeat until all unique pairs are checked.
- Display min_dist.
- Stop.

PSEUDO CODE:

ALGORITHM ClosestPair(points)

min_dist $\leftarrow$ infinity

n $\leftarrow$ length(points)

FOR i $\leftarrow$ 0 TO n - 2 DO

 x1 $\leftarrow$ points[i].x

 y1 $\leftarrow$ points[i].y

 FOR j $\leftarrow$ i + 1 TO n - 1 DO

 x2 $\leftarrow$ points[j].x

 y2 $\leftarrow$ points[j].y

 dist $\leftarrow \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$

 IF dist < min_dist THEN

 min_dist $\leftarrow$ dist

 END IF

 END FOR

END FOR

RETURN min_dist

END ALGORITHM

Time complexity : $O(n^2)$